COMPLEXITY ANALYSIS

Interview এর জন্য Complete Guide

6 Parts • 30+ Slides • Blog Series Recap

Part 1

Time Complexity

তোমার Program কত সময় নিচ্ছে?

কেন Complexity Analysis শিখবে?

Interview Scenario

"What's the time complexity?"

তুমি চুপ। মাথা ফাঁকা। পেটে চিনচিন।

"Can you optimize it?"

এই situation যেন না হয়,
তার জন্যই এই lecture।

Real-World Impact

Database: 10 লাখ row → efficient = ms, bad = 30s

Scalability: 1K user ok, 1M user = crash

Mobile: সীমিত memory → bad code = hang

Interview: FAANG → complexity is non-negotiable

গল্প দিয়ে শুরু — Harry Potter খুঁজতে হবে!

ঘরে n টা বই এলোমেলোভাবে জমানো। খুঁজে বের করো "Harry Potter"!

কী করবে?

একটা একটা বই তুলে দেখবে —
"এটা Harry Potter? না।"
"এটা? না।"
"এটা? হ্যাঁ!"

যতক্ষণ না পাও, দেখতেই থাকো।

Worst case: সবগুলো n টা বই দেখতে হবে → n steps!

3 টা Scenario

Best Case: উপরেই আছে → 1 step

Average Case: মাঝামাঝি → ~n/2 steps

Worst Case: একদম শেষে → n steps

n=10 → worst 10 বই দেখা
n=100 → worst 100 বই দেখা
n=10,000 → worst 10,000 বই!

বই বাড়লে কাজও বাড়ে — সোজাসাপ্টা !

Number Guessing Game — বোকা vs চালাক

1 থেকে 1000 এর মধ্যে আমি একটা number ভেবেছি। তুমি guess করো!

বোকা Approach

1, 2, 3, 4, 5... একটা একটা try

আমি ভেবেছি 998?
→ তোমার 998 টা guess লাগবে!

Worst case: 1000 guess
(পুরো range cover)

চালাক Approach — অর্ধেক বাদ!

500 বললে → "কম" → 500 টা বাদ!
250 বললে → "বেশি" → আবার অর্ধেক বাদ!

1000→500→250→125→63→32→
16→8→4→2→1

মাত্র ~10 guess!

Range	বোকা (step)	চালাক (step)
1 → 1,000	**1,000**	**~10**
1 → 10,00,000	**10,00,000**	**~20**

কেন সেকেন্ড দিয়ে মাপি না? — Variable n দিয়ে কেন?

কারণ ১: Input Size Change হয়

আজকে n = 100 data
কাল n = 1 লাখ
পরশু n = 1 কোটি

"500 টা operation" বললে — কোন n এর জন্য? n change হলে তো সব বদলায়!

তাই n এর ভিত্তিতে বলি।

কারণ ২: Machine Speed আলাদা

Machine A: 1 কোটি op/sec → 1 sec
Machine B: 1 কোটি op/5 sec → 5 sec

Same code, same n = 1 কোটি
কিন্তু সময় আলাদা!

"1.5 sec" শুধু তোমার machine এ সত্য,
"n টা operation" সব machine এ সত্য!

তাহলে কী করি?

সেকেন্ড দিয়ে মাপি না → machine dependent!

Input size n এর ভিত্তিতে operation সংখ্যা দিয়ে মাপি → machine independent!

"n টা operation" → সব machine এ সত্য। এটাই Big O এর core idea!

Big O Notation — পুরো Game এর Foundation

O(n) মানে:

Worst case এও n operation এর বেশি লাগবে না।
Upper bound — একটা guarantee!

"30 min এ পৌঁছাবো" → 15 min এও পারি,
কিন্তু 30 min cross হবে না। Big O ও তাই!

O(n)

Upper Bound (Worst Case)

Big O Rules

Rule	Example
Rule 1: Constants Drop	$O(2n) \to O(n)$, $O(100) \to O(1)$
Rule 2: Dominant Term	$O(n^2 + n) \to O(n^2)$
Rule 3: Different Inputs	a, b আলাদা → $O(m \times n)$, $O(n^2)$ না
Rule 4: Sequential = Add	$O(n) + O(n^2) \to O(n^2)$

O(1) — Constant Time

n = 100 হোক বা 1 কোটি — operation সবসময় fixed!

Problem: 1 + 2 + ... + n বের করো (Formula দিয়ে)

```
int sum = n * (n + 1) / 2;
cout << sum;  // একটা line!
// 3 operations — n যাই হোক → O(1)
```

O(1) মানে?

Operation count কোনো variable এর উপর depend করে না। সবসময় same!

3 ops, 30 ops — যাই হোক, fixed হলেই O(1)।

n	Loop — O(n)	Formula — O(1)
100	100 ops	**3 ops**
1,00,000	1 লাখ ops	**3 ops**
1,00,00,000	1 কোটি ops	**3 ops**

Same কাজ, same answer — আকাশ-পাতাল performance পার্থক্য!

O(n) — Linear Time

n বাড়লে operation সমান হারে বাড়ে — সোজাসাপ্টা!

Example 1: 1 থেকে n যোগফল (Loop)

```
int sum = 0;
for (int i = 1; i <= n; i++)
    sum += i;
// Loop n বার → O(n)
```

Example 2: Linear Search

```
for (int i = 0; i < n; i++)
    if (arr[i] == target)
        return i;
// Worst: n checks → O(n)
```

Example 3: 26 × n = O(n), O(26n) না!

```
for (char c = 'a'; c <= 'z'; c++) {   // 26 বার (fixed!)
    for (int i = 0; i < n; i++) {    // n বার
        if (s[i] == c) count[c]++;
    }
}  // 26 × n → constant 26 drop → O(n)
```

26 হলো constant factor!

n এর সাথে বাড়ে না — সবসময় 26।
তাই Big O তে বাদ → O(n)।

যদি বাইরের loop n বার ঘুরতো → $O(n^2)$

$O(n^2)$ — Quadratic Time

Nested loop → গুণ করো! দুটো loop n বার → $n \times n = n^2$

Example 1: All Pairs Print

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    cout << arr[i] << "," << arr[j];
// n × n = n² pairs → O(n²)
```

Example 2: Two Sum (LeetCode #1)

```
for (int i = 0; i < n; i++)
  for (int j = i+1; j < n; j++)
    if (arr[i]+arr[j] == target)
      return {i,j}; // O(n²)
```

Exact count: n(n-1)/2

Expand: $n^2/2 - n/2$
½ constant factor → drop
n/2 smaller term → drop
→

$O(n^2)$

n=5 → 10 checks

n=1000 → 4,99,500

n=10,000 → ~5 কোটি!

Rule: n বড় হলে n^2 এর কাছে n তুচ্ছ!

Interview Tip: Two Sum $O(n^2)$ → HashMap দিয়ে O(n) করা যায়! Trade-off: space O(n)।

$O(n^3)$ — Cubic Time

3 nested loops → $n \times n \times n = n^3$

Example: Hill Triplets — arr[i] < arr[j] > arr[k]

```
// মাঝেরটা দুই পাশের চেয়ে বড় — পাহাড়ের মতো
int count = 0;
for (int i = 0; i < n; i++)
  for (int j = i+1; j < n; j++)
    for (int k = j+1; k < n; k++)
      if (arr[i]<arr[j] && arr[j]>arr[k])
        count++;
// 3 nested loops → O(n³)
```

Polynomial Family:

1 loop → $O(n)$
2 nested → $O(n^2)$
3 nested → $O(n^3)$

$3n^3 + 2n^2 + n + 5$ → Big O তে শুধু সবচেয়ে বড় ঘাত: $O(n^3)$। বাকি সব তুচ্ছ!

n=100 → 10 লাখ ops
n=1000 → 100 কোটি!

O(√n) — Square Root Time

Problem: n এর কতগুলো divisor আছে?

Key Insight: Divisors pair এ আসে! 12 = 1×12, 2×6, 3×4

ছোট divisor সবসময় √n এর মধ্যে! √12 ≈ 3.46 → ছোটগুলো: 1, 2, 3

```
int countDivisors(int n) {
    int count = 0;
    for (int i = 1; i*i <= n; i++) { // √n পর্যন্ত
        if (n % i == 0) {
            count++;              // i divisor
            if (i != n/i) count++;// pair ও divisor
        }
    }
    return count;
} // Loop √n বার → O(√n)
```

n = 10^5 হলে:

Naive: 1,00,000 ops

Smart: ~316 ops

300× faster!

O(n × k) — যখন দুটো Variable

Problem: n জন student, k টা subject — প্রত্যেকের total marks

```
int marks[n][k];
for (int i = 0; i < n; i++) {       // n students
    int total = 0;
    for (int j = 0; j < k; j++)     // k subjects
        total += marks[i][j];
    cout << total;
}  // n × k → O(n × k)
```

n আর k আলাদা!

n=1000, k=5 ⟶ 5,000
n=1000, k=50 ⟶ 50,000

⚠️ **Interview Mistake: দুটো আলাদা input size কে n ধরে $O(n^2)$ বলে ফেলো — সাবধান!**

O(log n) — Logarithmic Time

প্রতি step এ problem অর্ধেক → Binary Search!

```
int binarySearch(vector<int>& arr, int t) {
    int l = 0, r = arr.size()-1;
    while (l <= r) {
        int m = (l+r)/2;
        if (arr[m] == t) return m;
        else if (arr[m] < t) l = m+1;
        else r = m-1;
    }
    return -1;
} // প্রতিবার অর্ধেক বাদ → O(log n)
```

log n মানে কী?

n কে কতবার 2 দিয়ে ভাগ করলে 1 হয়
16→8→4→2→1 = 4 বার → $\log_2 16 = 4$

Big O তে base matter করে না
$\log_2$ আর $\log_{10}$ — constant factor
($\log_2 n = 3.32 \times \log_{10} n$)

n	O(n) Linear	O(log n) Binary
1,000	1,000 বার	**~10 বার**
10 লাখ	10 লাখ বার	**~20 বার**
100 কোটি	100 কোটি বার	**~30 বার**

100 কোটি element এও মাত্র ~30 বার! Sorted data + Binary Search = 🔥

O(n log n) — Sorting Complexity

Most efficient sorting algorithms: Merge Sort, Quick Sort, Heap Sort → O(n log n)

```
sort(arr.begin(), arr.end());  // C++ STL
// O(n log n) — internally IntroSort
// n elements, each log n বার process
```

$O(n) < O(n \log n) < O(n^2)$
n=10 লাখ: $n=10^6$, $n \log n \approx 2\times10^7$, $n^2=10^{12}$

Sort + Binary Search = powerful combo!

$O(2^n)$ — Exponential Time

Interview Classic: Fibonacci Naive Recursive → DP Optimization

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}  // দেখতে সুন্দর, ভেতরে ভয়ংকর!
```

Recursive Tree:

প্রতিটা call → 2 টা call
fib(3) → 2 বার, fib(2) → 3 বার!

Overlapping subproblems!

fib(10): 177 fib(30): 26 লাখ+ **fib(50): ~1250 কোটি!**

DP দিয়ে Optimization: $O(2^n) \rightarrow O(n)$!

```
int dp[1000]; memset(dp, -1, sizeof(dp));
int fib(int n) {
    if (n<=1) return n;
    if (dp[n]!=-1) return dp[n]; // আগেই solve!
    return dp[n] = fib(n-1) + fib(n-2);
}
// save করো, পরেরবার saved answer দাও!
```

$O(2^n) \rightarrow O(n)$

fib(50): 1250 কোটি → **50!**

এটাই DP এর magic — overlapping subproblem চিনো, save করো, reuse করো!

O(n!) — Factorial Time

n টা element এর সব permutation বের করো

`{1,2,3} → 6 ways | {1,2,3,4} → 24 ways | {1,...,10} → 36,28,800!`

```
sort(nums.begin(), nums.end());
do {
    for(int x : nums) cout << x << " ";
} while(next_permutation(nums.begin(), nums.end()));
// n! permutations → O(n!)
```

n	n!	Status
3	6	⚡ instant
10	36,28,800	⚡ ok
15	~130 বিলিয়ন	❌ hours
20	$\sim 2.4 \times 10^{18}$	❌ impossible

সবচেয়ে ধীর complexity! n > 20 = practically impossible।

Complexity Ranking — Best to Worst

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Complexity	$n=10^6$ Operations	Rating
$O(1)$	1	⚡ Instant
$O(\log n)$	~20	⚡ Instant
$O(\sqrt{n})$	~1,000	✅ Very Fast
$O(n)$	10,00,000	✅ Good
$O(n \log n)$	~2,00,00,000	✅ Okay
$O(n^2)$	10^{12}	❌ Very Slow
$O(2^n)$	Atoms in universe	❌ Impossible
$O(n!)$	∞	❌ Forget it

n দেখেই বুঝে যাও — কোন Complexity চলবে

C++ ~10^8 ops/sec | Problem constraint ⟶ approach select!

n Range	Max Complexity	Approach
$n \le 10\text{-}12$	$O(n!)$	Brute force permutation
$n \le 20\text{-}25$	$O(2^n)$	Bitmask DP, backtracking
$n \le 500$	$O(n^3)$	Floyd-Warshall, DP
$n \le 10{,}000$	$O(n^2)$	Nested loops, simple DP
$n \le 10^6$	$O(n \log n)$	Sorting + binary search
$n \le 10^8$	$O(n)$	Single pass, two pointer
$n > 10^8$	$O(\log n)/O(1)$	Math formula, binary search

💡 Interview: constraint শুনেই জিজ্ঞেস করো — "What's the expected input size?"

Code Patterns → Complexity

Code Pattern	Complexity
Single for loop (0 to n)	O(n)
Nested 2 loops (both n)	$O(n^2)$
Triple nested	$O(n^3)$
Loop: i *= 2 or i /= 2	O(log n)
Outer n, inner log n	O(n log n)
Recursive: 2 calls per level	$O(2^n)$
Recursive: 1 call, n/2 size	O(log n)
Fixed loop × n loop	O(n) — fixed = constant!

Part 2

Space Complexity

তোমার Program কতটুকু Memory খাচ্ছে?

Auxiliary Space vs Total Space

Auxiliary Space (Extra)

Input বাদে তুমি নতুন কতটুকু memory নিচ্ছো। Interview তে এটাই জিজ্ঞেস করে!

Examples:
- int x → O(1)
- new array[n] → O(n)
- n×n grid → $O(n^2)$

Total Space

Input + Auxiliary — সব মিলিয়ে।
Specifically জিজ্ঞেস করলে বলো।

Palindrome Check:

```
// O(n) space: reverse string বানালে
string rev(s.rbegin(), s.rend());
return s == rev;
```

```
// O(1) space: two pointer
while (l < r)
  if(s[l++]!=s[r--]) return false;
```

Space Complexity — Examples

O(1) Space: Variables only — swap(a,b), findMax(arr)

O(n) Space: HashMap for duplicate check, freq array

O(log n) Space: Binary conversion — $n=10^5 \rightarrow$ মাত্র ~17 chars

$O(n^2)$ Space: 2D visited grid, DP table (LCS, Edit Distance)

Duplicate Check — O(n) space with HashMap

```
bool hasDuplicate(vector<int>& arr) {
    unordered_map<int,int> freq;
    for (int x : arr) {
        freq[x]++;
        if (freq[x]==2) return true;
    }
    return false;
} // freq map → worst n entries → O(n) space
```

Island Count — $O(n^2)$ space

```
// n×n grid + n×n visited
vector<vector<bool>>
  visited(n, vec(m,false));
// visited = n×m extra
// → O(n²) auxiliary
```

Recursion এর Hidden Space Cost

কোনো array বানাওনি — তবুও O(n) space! Stack frames!

```
int factorial(int n) {
    if (n<=1) return 1;
    return n * factorial(n-1);
}
```

factorial(5) → Stack:

frame 5: n=1 → return 1
frame 4: n=2 → waiting...
frame 3: n=3 → waiting...
frame 2: n=4 → waiting...
frame 1: n=5 → waiting...

5 frames একসাথে ! → O(n) space

Rule: Recursive space = maximum depth, total calls না!

Function	Time	Depth → Space
factorial(n)	O(n)	n → O(n)
fib(n) naive	$O(2^n)$	n → O(n) ← not 2^n!
binary search rec	O(log n)	log n → O(log n)
merge sort	O(n log n)	log n → O(log n) + O(n)

Time-Space Trade-off

সময় বাঁচাতে memory খরচ, memory বাঁচাতে সময় বেশি — balance!

কম Memory, বেশি সময়

Duplicate: nested loop
Time: $O(n^2)$ | Space: $O(1)$

n=10 লাখ → minutes!

বেশি Memory, কম সময়

Duplicate: HashMap
Time: $O(n)$ | Space: $O(n)$

n=10 লাখ → instant!

Part 3

Advanced Topics

Big O/Ω/Θ, Amortized Complexity

Big O vs Big Ω vs Big Θ

Big O	Big Ω	Big Θ
Upper Bound	**Lower Bound**	**Tight Bound**
Worst Case "এর বেশি লাগবে না"	Best Case "এর কম লাগবে না"	Both = Same "সবসময় এতটুকুই"
Linear Search: O(n)	**Linear Search: Ω(1)**	**Array traverse: Θ(n)**

Insertion Sort: Best Ω(n) — already sorted | Worst O(n^2) — reverse sorted | Θ নেই (best ≠ worst)

Interview তে 90% সময় Worst Case (Big O) জিজ্ঞেস করে!

Amortized Complexity

vector::push_back — O(1) amortized কীভাবে?

বেশিরভাগ push_back: O(1) — শুধু element বসাও

Capacity full হলে: RESIZE! → দ্বিগুণ array, সব copy → O(n)

কিন্তু resize বারবার হয় না — gap দ্বিগুণ বাড়ে → total n push = O(n) cost → per push O(1)!

	Average O(1)	Amortized O(1)
Guarantee?	না — luck dependent	হ্যাঁ — total always O(n)
Unlucky?	সব expensive হতে পারে	কয়েকটা expensive, বাকি cheap
Example	Hash Table lookup	vector::push_back

Part 4

NP Complete & NP Hard

কিছু Problem কেন "অসম্ভব"

P, NP, NP-Complete, NP-Hard

P	NP	NP-C	NP-H
Solve সহজ Polynomial Time	Verify সহজ Polynomial Time	NP এর কঠিনতম Solve = exponential	সবচেয়ে কঠিন Verify ও কঠিন
Sorting **Dijkstra** **Binary Search**	**Sudoku verify** **Knapsack verify**	**TSP** **Knapsack** **Graph Coloring**	**Halting Problem** **Chess Best Move**

P = NP?

50+ বছর ধরে unsolved! Solve করলে $1M পুরস্কার।
P=NP হলে → encryption crack, drug discovery রাতারাতি, optimization সব সহজ!

Part 5

Reference Tables

Algorithm & DS — Quick Reference

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	$O(n)$	✓
Quick	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$	$O(\lg n)$	✗
Heap	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	$O(1)$	✗
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓

Data Structure Operations

DS	Access	Search	Insert	Delete	কখন?
Array	O(1)	O(n)	O(n)	O(n)	Random access
HashMap	—	O(1)*	O(1)*	O(1)*	Fast lookup
BST balanced	O(lg n)	O(lg n)	O(lg n)	O(lg n)	Sorted order
Heap	O(1) top	O(n)	O(lg n)	O(lg n)	Min/Max fast
Stack	O(n)	O(n)	O(1)	O(1)	LIFO, DFS
Queue	O(n)	O(n)	O(1)	O(1)	FIFO, BFS

Graph Algorithms & Techniques

Algorithm	Time	কাজ
BFS/DFS	O(V+E)	Traversal, shortest unweighted
Dijkstra	O((V+E)lg V)	Shortest path, no neg
Bellman-Ford	O(V×E)	Shortest path, neg ok
Floyd-Warshall	$O(V^3)$	All-pairs shortest
Kruskal/Prim	O(E lg E)	MST

Technique Paradigms

Two Pointer — O(n)
Sorted pair search

Sliding Window — O(n)
Subarray max/min

Binary Search — O(lg n)
Sorted data

Greedy — varies
Locally optimal

DP — $O(n^2)$
Overlapping sub

Backtracking — $O(2^n)$
Explore & prune

Part 6

Practice Problems

Code দেখে Complexity বলো!

Practice Set 1

Q 1

```
for(i=1; i<=n; i*=2)
  print(i);
```

O(log n), Space O(1)

i doubles ⟶ log n iterations

Q 2

```
for(i=0;i<n;i++)
  for(j=0;j<i;j++)
    print;
```

$O(n^2)$, Space O(1)

0+1+...+(n-1) = n(n-1)/2

Q 3

```
j=0;
for(i=0;i<n;i++)
  while(j<n) j++;
```

O(n), Space O(1)

while total n বার, not n^2

Q 4

```
void f(int n) {
  if(n<=0) return;
  f(n-1); f(n-1);
}
```

$O(2^n)$, Space O(n)

2 calls/level, depth n

Practice Set 2

Q 5

```
for(i=n;i>0;i/=2)
  for(j=0;j<n;j++)
    print;
```

O(n log n), Space O(1)

outer log n, inner n

Q 6

```
int f(n) {
  if(n<=1) return 1;
  return f(n/2)+f(n/2);
}
```

O(n), Space O(log n)

2 calls but size halves

Q 7

```
set<int> s;
for(i=0;i<n;i++)
  s.insert(arr[i]);
// s.insert = O(log n)
```

O(n log n), Space O(n)

n inserts × log n each

Q 8

```
bool dp[n][n];
for(i...) for(j...)
  dp[i][j] = ...;
// 2D DP table
```

$O(n^2)$, Space $O(n^2)$

n×n table — time & space

Interview Tips — Final Weapons

1

Brute Force → Optimize

"$O(n^2)$ brute force দেখাই, তারপর HashMap দিয়ে $O(n)$ করি"

2

Concrete Numbers

"$n=10^5$ তে $O(n^2) = 10^{10}$ → ~100 sec. Too slow!"

3

Trade-off Discuss

"Time $O(n)$ but extra $O(n)$ space. Which preferred?"

4

n Range → Approach

"$n \leq 10^5$ means $O(n \log n)$ is fine. Let me use sorting."

Summary — কী শিখলে আজকে

Time: $O(1)$, $O(\log n)$, $O(\sqrt{n})$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$

Big O / Ω / Θ — Average vs Amortized

Sorting, DS, Graph — Reference Tables

Space: Auxiliary vs Total, Recursion Stack, Trade-off

NP Complete, NP Hard — P = NP?

Practice Problems — Code → Complexity

Happy coding & good luck with your interviews! 🚀